Computer Vision — CAT-1 Assignment Solutions

Worked answers to all nine questions, with calculations shown in full

Question 1 — Disparity and Depth from Stereo Correspondences

Given: baseline B = 0.20 m, focal length f = 1000 pixels. Coordinates are given as (x_L, x_R) — the horizontal pixel location of the same feature in the left and right images.

Disparity: $d = x_L - x_R$ *Depth:* $Z = (f \times B) / d$

Point	x_L	x_R	Disparity d (px)	Depth Z = fB/d (m)
A	450	420	30	6.67
B	610	570	40	5.00
C	760	735	25	8.00
D	900	850	50	4.00

Ordering nearest → farthest (smallest Z to largest Z):

D (4.00 m) → B (5.00 m) → A (6.67 m) → C (8.00 m)

Effect of a ±3-pixel disparity error

Since Z = fB/d, depth is a non-linear (inverse) function of disparity, so a fixed pixel error Δd = 3 does not produce a fixed depth error — it produces a much larger error for points with small disparity (far points) than for points with large disparity (near points). Differentiating, $\Delta Z \approx (fB/d^2)\cdot\Delta d = (Z^2/fB)\cdot\Delta d$, so the error grows with the **square** of the depth. Recomputing Z at d–3 and d+3 for each point:

Point	d (px)	Z at d–3 (m)	Z at d+3 (m)	Approx. error range
A	30	7.41 (d=27)	6.06 (d=33)	≈ –0.61 / +0.74 m
B	40	5.41 (d=37)	4.65 (d=43)	≈ –0.35 / +0.41 m
C	25	9.09 (d=22)	7.14 (d=28)	≈ –0.86 / +1.09 m
D	50	4.26 (d=47)	3.77 (d=53)	≈ –0.23 / +0.26 m

Analysis: Point C, the farthest point with the smallest disparity (25 px), suffers the largest depth uncertainty (about ±1 m), while point D, the nearest with the largest disparity (50 px), suffers the smallest uncertainty (about ±0.25 m). This illustrates a fundamental limitation of stereo vision: depth resolution degrades quadratically with distance, so stereo systems are accurate for near objects but increasingly imprecise for distant ones. A wider baseline or higher-resolution sensor (finer sub-pixel disparity estimation) reduces this error at any given depth.

Question 2 — Homography, Rectification, DLT, RANSAC

Any three of the four techniques, explained with examples and their role in alignment:

1. Homography

A homography is a 3×3 projective transformation matrix H that maps points on one plane to corresponding points on another image plane, x' = Hx (in homogeneous coordinates). It has 8 degrees

of freedom and is valid whenever the imaged scene is planar, or when the camera undergoes pure rotation about its optical centre. **Example:** stitching two overlapping photos of a flat wall or a printed document taken from slightly different angles — once H is estimated, one image can be warped onto the other's coordinate frame so that the overlapping regions align pixel-for-pixel. **Role:** it is the core geometric model used in panorama stitching, planar object tracking, and perspective (bird's-eye view) correction.

2. Image Rectification

Rectification re-projects a stereo image pair so that corresponding epipolar lines become horizontal and aligned on the same image row. It is computed from the fundamental/essential matrix (or known camera calibration) and applies a pair of homographies, one to each image. **Example:** before running block-matching stereo on a car's front-facing stereo camera, both frames are rectified so that a feature at row y in the left image only needs to be searched along row y in the right image. **Role:** it reduces the 2-D correspondence search of stereo matching to a fast 1-D search, which is essential for real-time depth estimation and 3-D reconstruction.

3. RANSAC (Random Sample Consensus)

RANSAC is a robust model-fitting algorithm that estimates a geometric model (e.g. a homography or the fundamental matrix) in the presence of outliers. It repeatedly draws a minimal random subset of point correspondences, fits the model, counts how many of the remaining matches agree with it within a tolerance (inliers), and keeps the model with the largest inlier support. **Example:** when matching SIFT/ORB features between two overlapping photographs, many matches are wrong (mismatched textures); RANSAC discards these outliers before the final homography is computed. **Role:** it makes homography and fundamental-matrix estimation reliable in real images where feature matching is always noisy.

> *(A fourth technique, the Direct Linear Transform (DLT), is the linear least-squares algorithm typically used to actually solve for H or F once a set of correspondences — ideally the RANSAC inlier set — is available; it arranges each correspondence into two linear equations and solves the resulting system via SVD.)*

Question 3 — The Fundamental Matrix and Epipolar Geometry

In a two-view stereo system, a 3-D point projects to points x (left image) and x' (right image). Epipolar geometry states that x' must lie on a specific line in the right image (the epipolar line) determined by x — it does not have to be searched over the whole image. This constraint is captured algebraically by the **fundamental matrix F**, a 3×3, rank-2 matrix satisfying the epipolar constraint:

$$x'^{T} \cdot F \cdot x = 0 \quad \textit{for every pair of corresponding points } (x, x')$$

Given a point x in the left image, the vector l' = Fx is the equation of its corresponding epipolar line in the right image; similarly $l = F^{T}x'$ is the epipolar line in the left image. All epipolar lines in an image pass through a common point called the epipole (the projection of the other camera's centre). Unlike the essential matrix, F requires no knowledge of the cameras' intrinsic parameters — it relates the two images purely in pixel coordinates, which is why it is central to uncalibrated stereo and structure-from-motion.

Estimating F from corresponding points

F is commonly estimated using the **eight-point algorithm**:

- Collect at least 8 corresponding point pairs (x_i, x'_i) between the two images (found via feature matching, then cleaned with RANSAC to remove outliers).

- Normalize the coordinates (translate and scale so points are centred at the origin with average distance $\sqrt{2}$ from it) — this greatly improves numerical stability (Hartley's normalization).
- Each correspondence gives one linear equation in the 9 unknown entries of F; stacking all correspondences gives a homogeneous linear system $A \cdot f = 0$, solved by taking the singular vector of A corresponding to the smallest singular value (via SVD).
- The resulting matrix generally has full rank 3, but F must have rank 2 (since epipolar lines all meet at the epipole); this is enforced by taking the SVD of the estimated F, zeroing its smallest singular value, and reconstructing F.
- Finally, the normalization transform is undone to obtain F in the original pixel coordinates.

Question 4 — Computer Vision for Criminal Detection via CCTV

Computer vision extends beyond simply enhancing a video frame — it enables a surveillance system to understand and act on what is happening in the scene. Typical applications supporting criminal detection include:

- **Face recognition** — matching detected faces in CCTV footage against a watch-list database to identify known suspects.
- **Object/weapon detection** — deep-learning detectors (e.g. YOLO-style models) flagging weapons, unattended bags, or forbidden items in real time.
- **Anomaly and behaviour detection** — learning normal crowd/traffic patterns and automatically raising alerts for loitering, running, fighting, or a person entering a restricted zone.
- **License-plate recognition (ANPR)** — reading vehicle number plates for tracking getaway vehicles.
- **Person re-identification and tracking** — following the same individual as they move across multiple non-overlapping camera views, reconstructing their path through a building or city.
- **Automated alerting** — pushing real-time notifications to control-room operators instead of requiring someone to watch dozens of screens continuously.

Image processing vs. computer vision on the same surveillance frame

Aspect	Image Processing	Computer Vision
Goal	Improve/transform the image itself	Interpret and understand image content
Typical operations	Contrast/brightness enhancement, denoising, edge detection, sharpening for a low-light frame	Detect and classify faces, people, weapons; recognise actions/behaviour
Output	A cleaner image for a human to view	Structured decisions: identity, alert, bounding boxes, track ID
Automation level	Passive — still needs a human to interpret it	Active — can trigger an automatic alarm/response
Example on the same frame	Sharpen a blurry night-time frame so a guard can read the number plate	Automatically detect the vehicle, OCR the plate, and match it to a stolen-vehicle database

In short: image processing prepares or enhances the pixels; computer vision goes further and extracts meaning from them, which is what allows a surveillance system to actively assist in detecting and responding to criminal activity rather than only recording it.

Question 5 — Convolution Output Size (Stride = 1, No Padding)

> **Note on the given data:** the source question refers to "the image" but the input matrix/kernel was not included with the question text in the uploaded file. Below is the general method and formula, worked through on a representative 5×5 input with a 3×3 kernel — substitute your actual matrices into the same procedure to get the specific numeric answer.

General formula for the output size

$$\text{Output size} = \lfloor (N - K) / S \rfloor + 1$$

where N is the input matrix size (N×N), K is the kernel size (K×K), and S is the stride. With stride S = 1 and no padding, this simplifies to Output size = N – K + 1.

Worked example (illustrative)

Input matrix (5×5):

1	2	3	0	1
0	1	2	3	1
1	0	1	2	0
2	1	0	1	3
0	2	1	0	1

Kernel (3×3):

1	0	-1
1	0	-1
1	0	-1

With N = 5, K = 3, S = 1: Output size = 5 – 3 + 1 = 3, i.e. a 3×3 output matrix. Each output cell is the sum of the element-wise product between the 3×3 kernel and the corresponding 3×3 window of the input, sliding the window one pixel at a time (stride 1) with no padding at the border.

Resulting 3×3 output matrix:

-4	-4	1
1	-3	-2
2	-1	1

Apply exactly this sliding-window, element-wise-multiply-and-sum procedure to your actual image and kernel values to get the specific output matrix asked for.

Question 6 — Orthogonal vs Euclidean vs Affine Transformations

Transformation	What it allows	Properties preserved	DoF
Orthogonal	Rotation and/or reflection about the origin (a linear map with an orthogonal matrix, det = ±1)	Lengths, angles, and areas	1 (rotation angle)
Euclidean (rigid)	Rotation + translation	Lengths, angles, parallelism, areas (an isometry)	3 (θ, t_x, t_y)

Affine	Rotation + translation + non-uniform scaling + shear	Parallelism and ratios of lengths along a line only; angles and absolute lengths are NOT preserved	6

Which transformation corrects a rotated, stretched and skewed document?

An **affine transformation** is the right choice. Reasoning: an orthogonal transformation only rotates or reflects, so it cannot correct stretching or skew. A Euclidean transformation adds translation but is still a rigid-body motion — it preserves angles and lengths exactly, so it also cannot fix stretching (non-uniform scale) or skew (shear), both of which change angles and relative lengths. An affine transformation, however, explicitly includes independent scaling along each axis and a shear component in addition to rotation and translation, so it can simultaneously undo rotation, unequal stretching, and skew while still keeping straight lines straight and parallel lines parallel — exactly the distortions described (a document warped by scanning/photographing at a slight angle typically distorts this way, as long as true perspective foreshortening is negligible; if the distortion also converges towards a vanishing point, a full projective/homography correction would additionally be needed).

Question 7 — Relating Two Views of a Planar Surface

When two images capture the same planar surface from different viewpoints, the corresponding points are related by a **homography**, a 3×3 projective matrix H such that, in homogeneous coordinates, $x'_i = H \cdot x_i$ for every pair of corresponding points. H has 8 independent parameters and can be solved linearly (via the DLT) given at least 4 non-collinear point correspondences, typically refined and freed of outliers using RANSAC.

Conditions under which the relationship holds

- The imaged scene is (approximately) planar — e.g. a document, a wall, a floor, or a whiteboard; or
- The camera undergoes pure rotation about its optical centre with no translation (as in panorama capture), even if the scene is not planar.
- If neither condition holds — the scene has real 3-D depth variation and the camera also translates — a single homography cannot correctly relate all points, and full epipolar geometry / 3-D reconstruction is required instead.

Uses

- **Image stitching** — warping overlapping photos into one another's frame to build seamless panoramas.
- **Perspective correction** — un-warping a photographed plane back to a fronto-parallel view.
- **Document scanning** — apps such as mobile scanner tools detect the four corners of a document and apply a homography to produce a flat, rectangular scan.
- **Planar-object tracking** — tracking a flat marker/logo/poster across video frames by continually re-estimating H between the reference image and each new frame.

Question 8 — Histogram Equalization

Given image (3×3), 9 pixels:

5	3	-2
0	1	4

2	-1	0

The image contains 8 distinct intensity levels: –2, –1, 0, 1, 2, 3, 4, 5 (taking L = 8 levels, so the equalized output range is rank 0 to L–1 = 7). Step 1: build the frequency table, cumulative frequency (CDF), and map each level using s = round[(L–1) × CDF(r) / N], with N = 9 total pixels.

Level r	-2	-1	0	1	2	3	4	5
Frequency	1	1	2	1	1	1	1	1
Cumulative freq.	1	2	4	5	6	7	8	9
s = round(7×CDF/9)	1	2	3	4	5	5	6	7

Step 2: replace every original pixel by its mapped value s:

7	5	1
3	4	6
5	2	3

This is the histogram-equalized image. The transformation has stretched the intensity distribution so it spans the full available range (0–7) more evenly than the original, increasing overall contrast — the essence of histogram equalization.

> (Note: the original values include negatives, which is unusual for pixel intensities; the calculation above treats the 8 distinct values present as the ordered intensity levels of the image, which is the standard approach when an explicit bit-depth/range is not otherwise specified.)

Question 9 — Monocular vs Stereo Depth Estimation

Yes — determining the actual (metric) distance of an object from a single perspective image is fundamentally difficult. A perspective projection collapses the 3-D world onto a 2-D plane, so infinitely many 3-D points along the same ray project to the same image pixel (an object could be small-and-near or large-and-far and look identical in the image). This is the classic **scale ambiguity** of monocular vision: a single image gives only relative depth cues, not an absolute metric distance, unless extra information (known object size, camera height, learned statistical priors, etc.) is supplied.

Monocular depth cues

- Perspective convergence (parallel lines appearing to meet at a vanishing point), relative size of known objects, texture-gradient density, occlusion (nearer objects block farther ones), shading/shadows, and motion parallax across a video sequence.
- Modern deep-learning monocular depth estimators (e.g. trained CNNs) learn these statistical cues from large datasets to predict a plausible depth map from one image, but the estimate remains scale-ambiguous/relative unless calibrated against known references.

Stereo depth estimation

- Uses two calibrated cameras with a known baseline B; depth is computed geometrically by triangulation from the disparity d between matched points, Z = fB/d (as used in Question 1).
- This gives a true metric depth directly from geometry, without needing learned priors, and is generally more accurate for well-textured scenes.

- Limitations: requires reliable correspondence matching (fails on textureless or repetitive surfaces), needs accurate calibration/rectification, and accuracy degrades quadratically with distance (fine disparity errors cause large depth errors far away, as shown in Question 1).

Example comparison

A single dashcam photo of a car ahead cannot tell you its exact distance in metres without knowing the car's real-world size in advance (monocular ambiguity); a stereo dashcam rig with two synchronized cameras can directly triangulate the car's distance from the disparity between the two views, giving a precise metric measurement — which is why autonomous-driving and robotics systems that need reliable metric depth typically favour stereo (or LiDAR) over pure monocular vision, using monocular deep-learning depth mainly as a complementary or lower-cost cue.